

Embedded Linux & System Software Interview Handbook

Chapter 4 - Board Bring-up (Part 4)

Topics: Device Tree, Linux Kernel Bring-up, Driver Probing and Boot Failure Analysis

4.17 Device Tree (DT)

Modern ARM-based Linux systems use a Device Tree (DT) to describe the hardware. The Linux kernel is generic—it does not know the board-specific peripherals until it parses the Device Tree Blob (DTB).

Why Device Tree?

Instead of recompiling the kernel for every hardware variant, Linux uses a Device Tree to describe CPU, memory, GPIOs, I²C buses, SPI controllers, UARTs, Ethernet, USB, regulators and clocks.

Typical Device Tree Structure

```
/ {
memory { ... }
cpus { ... }
chosen { bootargs = ...; }
aliases { serial0 = &uart0; }
soc {
uart@... { status = "okay"; };
i2c@... { status = "okay"; };
spi@... { status = "disabled"; };
}
}
```

Interview Question

Q: Why is Device Tree required?

A: It separates hardware description from kernel code, allowing one kernel image to support multiple boards.

4.18 Linux Kernel Initialization

Once U-Boot loads the kernel and DTB into memory, it transfers execution to the kernel entry point. The kernel initializes memory management, scheduler, interrupts, drivers and eventually starts the init process.

Kernel Initialization Flow

Start Kernel → MMU → Scheduler → Interrupts → Driver Initialization → Mount RootFS → init/systemd → User Space

4.19 Driver Probing

Linux drivers are initialized through the driver model. During boot, the kernel matches Device Tree nodes with registered drivers. When a match is found, the driver's **probe()** function is invoked.

Driver Probe Responsibilities

- Allocate resources
- Map registers
- Request IRQs
- Enable clocks
- Initialize hardware
- Register device with kernel

Example

If a UART node is marked **status = "disabled"** in the Device Tree, the UART driver will never probe, even if the hardware is present.

4.20 Understanding Kernel Boot Logs

Kernel logs provide valuable information during bring-up. Engineers use the last successful log message to identify where boot stopped.

Last Log Seen	Likely Cause
Starting kernel...	Kernel image or entry point issue
Freeing unused kernel memory	RootFS problem
VFS: Unable to mount root fs	Wrong root device
Waiting for root device	Storage driver missing
Kernel panic	Driver or memory issue
No console output	UART/Clock/Power problem

Practical Debugging Scenario

Problem: Board hangs after 'Starting kernel...'

Possible causes:

- Incorrect Device Tree
- Wrong kernel load address
- MMU configuration issue
- DDR corruption
- Early kernel crash

Recommended approach:

1. Enable earlycon.
2. Verify DTB.
3. Check bootargs.
4. Compare with a known working board.
5. Validate DDR stability.

Real Interview Scenario

An interviewer may ask: 'Linux hangs after mounting the root filesystem. How would you debug it?' Discuss kernel logs, rootfs configuration, storage drivers, init process, and Device Tree configuration in a structured manner.

Common Interview Questions

1. What is a Device Tree?
2. Why was Device Tree introduced?

3. What is driver probing?
4. What does probe() do?
5. Why would a driver never probe?
6. How do you debug a board hanging after 'Starting kernel...'?
7. What are bootargs?

Key Takeaways

Device Tree is the bridge between hardware and the Linux kernel. A strong understanding of DT, driver probing, kernel logs and boot failures is essential for senior embedded Linux interviews.